

EXERCISES TO COMPLETE

Exercise 2 – Expression Tree

You should begin this task by downloading the [expression-tree.zip](#), extract the project and use the Eclipse (Import Existing Projects into Workspace) to import the project into your Eclipse workspace

The expression-tree mini-project is an object-oriented model of mathematical expressions, such as $x + (5 / y)$. The mini-project consists of two packages:

- `bcu.changeme.expressiontree` contains the model classes for the expression tree.
- `bcu.changeme.expressiontree.test` contains three unit test classes.

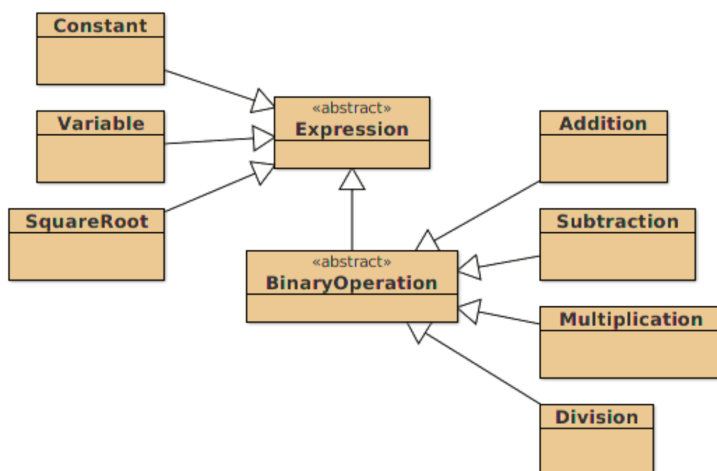
Once you have imported the mini-project, rename the `changeme` subpackage to a unique name of your choice which will identify the work as yours.

You should read the code given to you carefully to understand what it does. The unit tests may be the most informative.

You have been provided with the abstract classes, `Expression` and `BinaryOperation`. The `Expression` class has two methods which your subclasses will override:

- `evaluate` — evaluates the expression, given a Map of variables' values. For example, with $\{x=7, y=2.5\}$, the expression $x + (5 / y)$ evaluates to $7 + (5 / 2.5) = 9$.
- `freeVariables` — returns a set containing the names of the “free variables” in the expression. For example, the expression $x + (5 / y)$ has free variables x and y . The set returned must be unmodifiable.

The completed expression-tree mini-project will have all of the classes in the hierarchy shown below.



Your task is to complete the remaining model classes. These classes must be written in the `expressiontree` subpackage.

The 'Constant' subclass

The Constant class models a constant, such as 5 in the expression $x + (5 / y)$. This class should be a subclass of Expression.

- The class will need a field for the constant's value (as a double), and a constructor which sets this value.
- The evaluate method should simply return the constant's value.
- A constant has no free variables, so the freeVariables method should return an (unmodifiable) empty set.

The 'Variable' subclass

The Variable class models a variable, such as x in the expression $x + (5 / y)$. This class should be a subclass of Expression.

- The class will need a field for the variable's name (as a string), and a constructor which sets this value.
- The evaluate method should return the variable's value from the map. However, if the variable's name is not in the map, an IllegalArgumentException should be thrown.
- The freeVariables method should return an (unmodifiable) set containing only this variable's name.

The 'BinaryOperation' subclasses

The abstract subclass BinaryOperation models the “binary operations” +, -, * and /.

These are called “binary operations” because they have two operands — on the left and the right of the operator. These operands are themselves expressions, so a BinaryOperation is evaluated by first evaluating the left and right sub-expressions, and then performing the appropriate operation on their results.

To simplify development of its subclasses, the BinaryOperation class implements the evaluate and freeVariables methods from the Expression superclass. To complete the four classes Addition, Subtraction, Multiplication and Division, you must:

- Declare each class as a subclass of BinaryOperation.
- Declare a constructor which passes the left and right subexpressions to the superclass's constructor.
- Implement the evaluate(double a, double b) method which evaluates the operation, given the operands a and b.

The 'SquareRoot' subclass

The SquareRoot class models a square root operation, such as $\sqrt{1 + x}$. A square root expression has a sub-expression such as $1 + x$, which must be evaluated before the square root can be taken.

To complete the SquareRoot class, you must:

- Declare the class as a subclass of Expression.
- Declare a field to hold the sub-expression (as an Expression), and a constructor which sets this value.
- Implement the evaluate method by first evaluating the sub-expression, then returning the square root of its result.
- Implement the freeVariables method by simply returning the sub-expression's free variables.

Extension task

Override the built-in toString method on your classes, to render the expression tree as a string. Using the following tree as an example:

```
new Addition(  
    new Multiplication(  
        new Constant(4.5),  
        new Variable("x")  
    ),  
    new SquareRoot(  
        new Variable("y")  
    )  
)
```

For this example, the result as a string should be `"((4.5 * x) + sqrt(y))"`.

- Start with the Constant and Variable classes; their toString methods should simply return the constant's value and the variable's name, respectively.
- The SquareRoot subclass's toString method should return `"sqrt(...)"`, where ... is the sub-expression as a string.

The BinaryOperation subclasses are trickier, because left and right are `private` — the other classes cannot access these fields.

- Declare an `abstract` method named getSymbol in the BinaryOperation class, returning a string.
- Implement this new method in each subclass; for example, the Addition class's getSymbol method should return `"+"`.
- Now you can override toString in the BinaryOperation class. It should generate a string in the form `"(... + ...)"` where ... are the left and right sub-expressions as strings, and the getSymbol method is used to get the symbol in the middle.